K-NN classifier and Nearest Centroid Classifier

Contents

Summary

The present project examines the performance of the k-Nearest Neighbors (k-NN) and Nearest Centroid algorithms on the MNIST dataset. Through experimental trials with different values of the hyperparameter k and different distance metrics (Euclidean, Manhattan), the phenomenon of underfitting is analyzed and the generalization ability of the two models is compared.

Description of Algorithms

- **K-NN classifier - k nearest neighbors**

  The 'k-nearest neighbors (k-NN)' algorithm is a method used for both classification and regression tasks. Its main purpose is to predict the class of a new point by finding its k nearest neighbors in the training dataset. In the present project, we will focus exclusively on its use for digit classification.

  In the k-NN algorithm, the training dataset consists of data points, where each point is represented by a set of features and belongs to a specific class. When a new data point is given, the algorithm searches for its k nearest neighbors based on some distance metric, measuring similarity or dissimilarity in the feature space. Two of the most common metrics for feature vectors $x, y \in \mathbb{R}^n$ are the Euclidean distance and the Manhattan distance:

  Euclidean Distance:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

  Manhattan Distance:

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

  Once the k nearest neighbors are identified, the algorithm assigns the class to the new data point based on the majority vote of the classes of its neighbors.

  The parameter k determines the number of neighbors to be taken into account.

It is an important hyperparameter that must be chosen correctly to achieve the best performance of the algorithm. On the one hand, a small value of k can lead to overfitting, where the algorithm becomes sensitive to noise and can lead to poor generalization. On the other hand, a large value of k can lead to underfitting, where the algorithm may fail to capture the underlying patterns in the data.

The advantages of the algorithm are the following:

1. It is simple and understandable, making it a good and easy choice.
2. It makes no assumptions regarding the underlying distribution of the data, making it a non-parametric method. Thus, it has the ability to handle a wide range of data types and distributions.
3. It is suitable for both classification and regression tasks, making it flexible in its applications.

However, this algorithm also has some disadvantages:

1. It is computationally expensive during the prediction phase, especially when dealing with large datasets (such as the thousands of samples of MNIST). This happens because the algorithm must compute the distances between the new data point and all the training data points.
2. It is sensitive to the choice of the distance measurement. Different distance measurements may yield different results and the choice of measurement depends on the specific problem and the characteristics of the data.

- **Nearest Centroid classifier - Nearest centers**

  The 'Nearest Centroid classifier' algorithm is arguably one of the simplest classification algorithms in Machine Learning and operates on a simple principle: given an unknown data point, the Nearest Centroid classifier simply assigns to

it the class of the training sample whose mean (or center) is closest to it.
The process followed by this algorithm can be explained in three steps:

1. The center (centroid) μ_c of each class c is calculated during training. This center is obtained as the mean of all training vectors x belonging to the said class C_c:

$$\mu_c = \frac{1}{|C_c|} \sum_{x \in C_c} x$$

2. After training, for any new unknown point x, the distances between the point x and the center of each class μ_c are calculated using the Euclidean distance:

$$d(x, \mu_c) = \sqrt{\sum_{i=1}^{n} (x_i - \mu_{c,i})^2}$$

   or the Manhattan distance:

$$d(x, \mu_c) = \sum_{i=1}^{n} |x_i - \mu_{c,i}|$$

3. Out of all calculated distances, the minimum is selected. The class $\hat{y}$ of the center to which the distance from the given point is the minimum is assigned to the given point:

$$\hat{y} = \arg\min_c d(x, \mu_c)$$

Like every algorithm, this one also presents specific advantages:

1. It is extremely fast during the prediction phase. Unlike k-NN which must compare the new sample with the entire dataset, the Nearest Centroid algorithm makes comparisons only with the centers of the classes (in this specific problem, only 10 comparisons).

2. It requires minimal memory, as it stores only the centers of the classes and not all training data.

However, it has significant disadvantages that limit its accuracy in complex problems:

1. It assumes that the data of each class form a "spherical" distribution around their center. If the data of a class (e.g., different ways of writing the digit '4') have a complex or asymmetric distribution, the mean does not correctly represent the class.
2. It is sensitive to extreme values, as a highly distorted image during training can "pull" the center far away from its ideal position.

Examples of correct classification

- **K-NN classifier - k nearest neighbors**

  In Figure 1, six random samples from the test set are presented, which the k-NN algorithm successfully classified[cite: 68, 69].

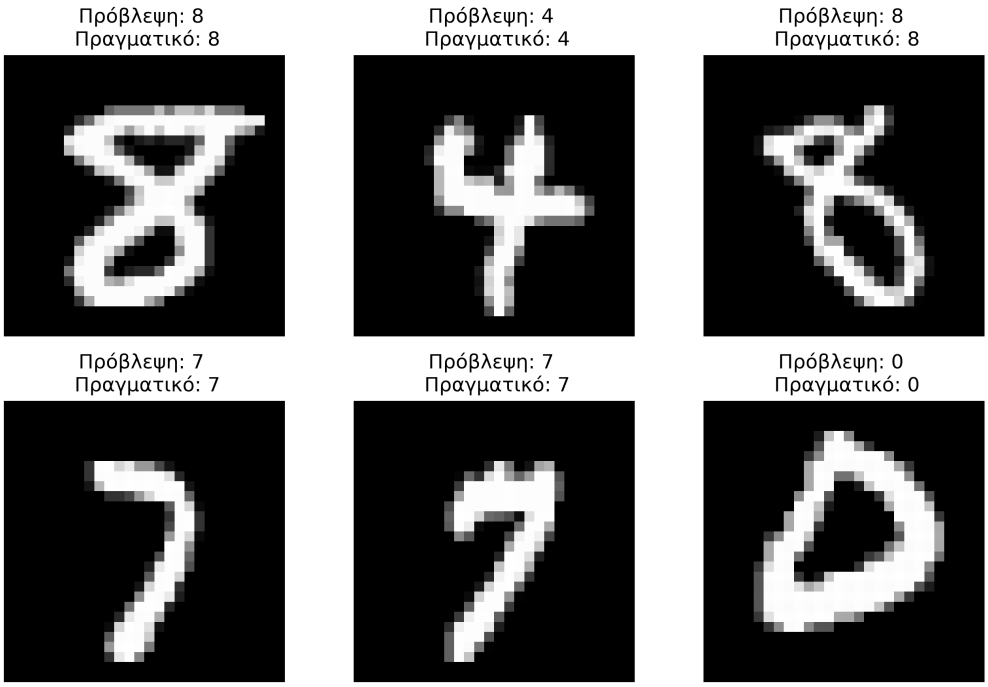

Figure 1: Samples of correct classification for the k-NN algorithm with $k = 3$ and Euclidean distance[cite: 78].

As observed, the algorithm recognizes with high accuracy digits that are clearly written and follow the typical forms of numbers[cite: 79]. The high similarity

between the feature vectors of these samples and their nearest neighbors in the training set allows the algorithm to reach the correct prediction[cite: 81].

- **Nearest Centroid classifier - Nearest centers**

  In Figure 2, we see examples where the Nearest Centroid algorithm achieved the correct prediction[cite: 83].

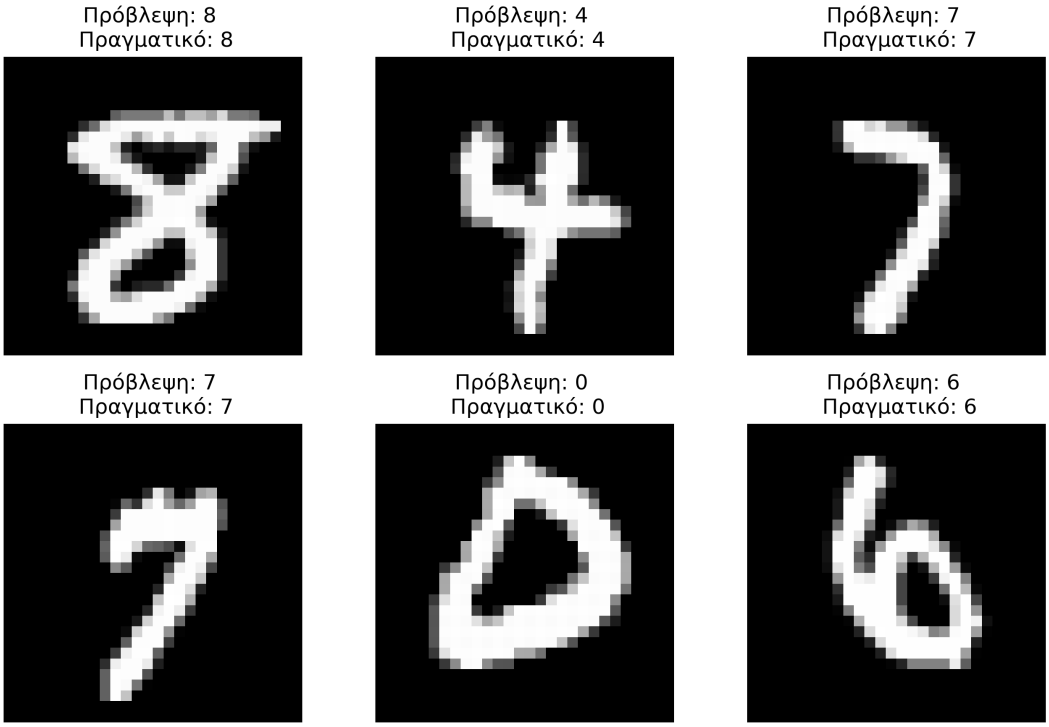

Figure 2: Samples of correct classification for the Nearest Centroid algorithm with Euclidean distance[cite: 93].

We observe that this algorithm, although simpler, performs well when the digits are "typical"[cite: 94]. That is, when their form is very close to the mean (the center) of their class[cite: 95].

Examples of incorrect classification

- **K-NN classifier - k nearest neighbors**

  Despite the high success rate, the algorithm committed some classification errors, which are presented in Figure 3.

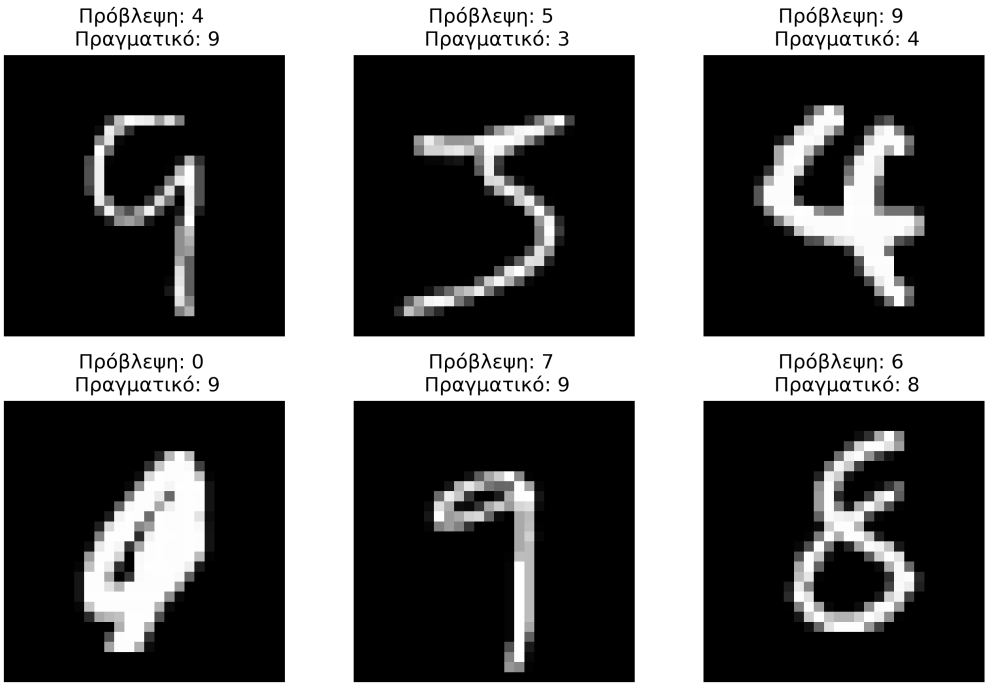

Figure 3: Samples of incorrect classification for the k-NN algorithm with $k = 3$ and Euclidean distance.

The analysis of the incorrect samples reveals that the failures of the algorithm are mainly due to:

- **Poorly written digits**: Some digits present heavy smudges or incomplete lines, resulting in them morphologically resembling other classes (e.g., a poorly written '4' can be considered a '9', as shown in the third image of the first row).
- **Particular slant or thickness**: Digits written with a large slant or unusual writing thickness distort the distances in the multidimensional feature space (as shown in the first image of the second row, where the model predicted the number '0' while the actual was '9').
- **Ambiguity between classes**: There are cases where even for the human eye the distinction is difficult, such as between '1' and '7' or '3' and '5'.

- **Nearest Centroid classifier - Nearest centers**

  In Figure 4, six samples are presented where the Nearest Centroid classifier failed to predict the correct class.

  The analysis of these errors confirms the limitations of the algorithm. In contrast to k-NN, which examines locally the closest points, Nearest Centroid relies on the mean of the entire class. Thus, if a digit is poorly written (e.g., a '7' that looks like a '1'), the algorithm classifies it into the class whose mean is morphologically closer to this specific sample.

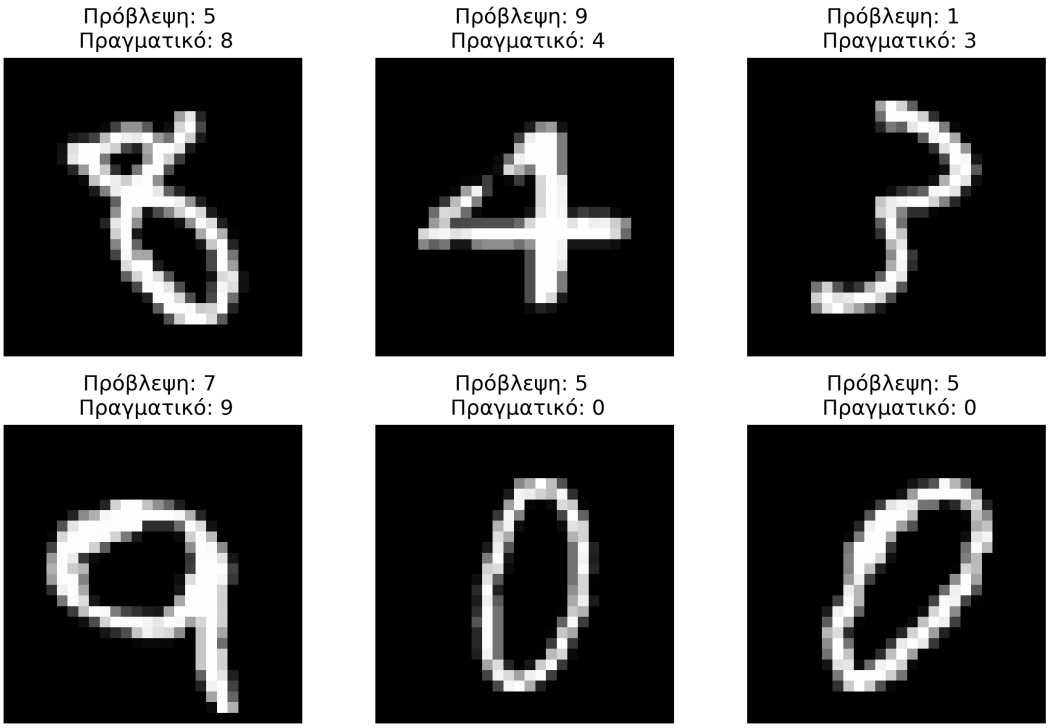

Figure 4: Samples of incorrect classification for the Nearest Centroid algorithm with Euclidean distance.

Results of Algorithms

In this section, the accuracy percentages of the implemented algorithms are presented. The evaluation was done on the test set which consists of 10,000 images.

Algorithm	**Hyperparameters**	**Accuracy**
k-NN (ready implementation)	$k = 3$, Euclidean	97.11%
k-NN (our implementation)	$k = 3$, Euclidean	97.30%
Nearest Centroid (ready implementation)	Euclidean	81.30%
Nearest Centroid (our implementation)	Euclidean	81.30%

Table 1: Comparative performance table of the classifiers.

Results Commentary: As shown in Table 1, the k-NN algorithm significantly outperforms the Nearest Centroid. This is expected, as k-NN examines the local structure of the data, whereas Nearest Centroid tries to simplify each class into a single representative point (the mean), thus losing information about the particular forms of the digits.

Of particular interest is the fact that our implementation of k-NN noted a slightly higher accuracy (97.30%) against the ready implementation of the scikit-learn library (97.11%). This difference can be attributed to the way ties are handled during the voting process of the neighbors, where the approach we followed proved slightly more effective for this specific sample of the MNIST dataset.

Confusion matrices

- **K-NN classifier - k nearest neighbors**

  1. $k = 3$, Euclidean: 97.30% [cite: 147]
  2. $k = 100$, Euclidean: 93.77% [cite: 148]
  3. $k = 2500$, Euclidean: 80.24% [cite: 149]
  4. $k = 9000$, Euclidean: 65.14% [cite: 150]
  5. $k = 34500$, Euclidean: 31.95% [cite: 151]
  6. $k = 60000$, Euclidean: 11.52% [cite: 152]
  7. $k = 3$, Manhattan: 96.59% [cite: 153]
  8. $k = 100$, Manhattan: 92.87% [cite: 154]
  9. $k = 2500$, Manhattan: 76.86% [cite: 155]
  10. $k = 9000$, Manhattan: 59.18% [cite: 156]
  11. $k = 34500$, Manhattan: 25.63% [cite: 157]
  12. $k = 60000$, Manhattan: 11.52% [cite: 158]

  If we compare the evaluation results for the various values of neighbors k and the two metrics (Euclidean distance and Manhattan distance), we observe two important phenomena[cite: 159]:

- **Underfitting Phenomenon**: In both metrics used, as k grows, the accuracy plummets[cite: 161].
  (In Euclidean distance from 97.30% to 11.52% [cite: 162]
  In Manhattan distance from 96.59% to 11.52%) [cite: 162]
- **Metrics Comparison**: The Euclidean distance performs slightly better than the Manhattan distance[cite: 163]. This is completely logical in the MNIST dataset, as the Euclidean distance better perceives the curves of handwritten digits, while the Manhattan distance loses some of the visual information[cite: 164].

In Figures 5 and 6, the confusion matrices for the k-NN algorithm using Euclidean distance and Manhattan distance, respectively, are presented for the various values of k[cite: 165]. The vertical axis represents the true number that the image depicted, while the horizontal axis represents the number that the algorithm predicted[cite: 166]. The diagonal represents the correct predictions[cite: 167]. The darker the color on the diagonal and the larger the numbers there, the better the model performed[cite: 167]. There, the Predicted label identical to the True label[cite: 168]. The remaining boxes show the wrong predictions of the algorithm[cite: 168].
Analyzing the form of the matrices, we clearly observe the progression of the underfitting phenomenon[cite: 169]. Specifically:

– **For** $k = 3$ **and** $k = 100$: The diagonal is intensely colored, indicating high accuracy[cite: 170]. The errors off-diagonal are minimal, which means that the algorithm correctly distinguishes the differences between the digits[cite: 171].

– **For** $k = 2500$ **and** $k = 9000$: The diagonal begins to "fade" and the errors increase[cite: 172]. Because the algorithm now examines excessively many

neighbors, it loses the ability to locate the local, detailed characteristics of each digit[cite: 172, 174].

– **For** $k = 60000$: The underfitting phenomenon reaches its peak[cite: 175]. In this extreme case, the algorithm consults the entire training set (all 60,000 images) for each new prediction[cite: 176]. As a result, it completely ignores the characteristics of the input and simply continuously selects the majority class of the set, which in this specific dataset happens to be the digit 1[cite: 177]. This is captured visually with the complete disappearance of the diagonal and the creation of an intense, vertical column[cite: 177].

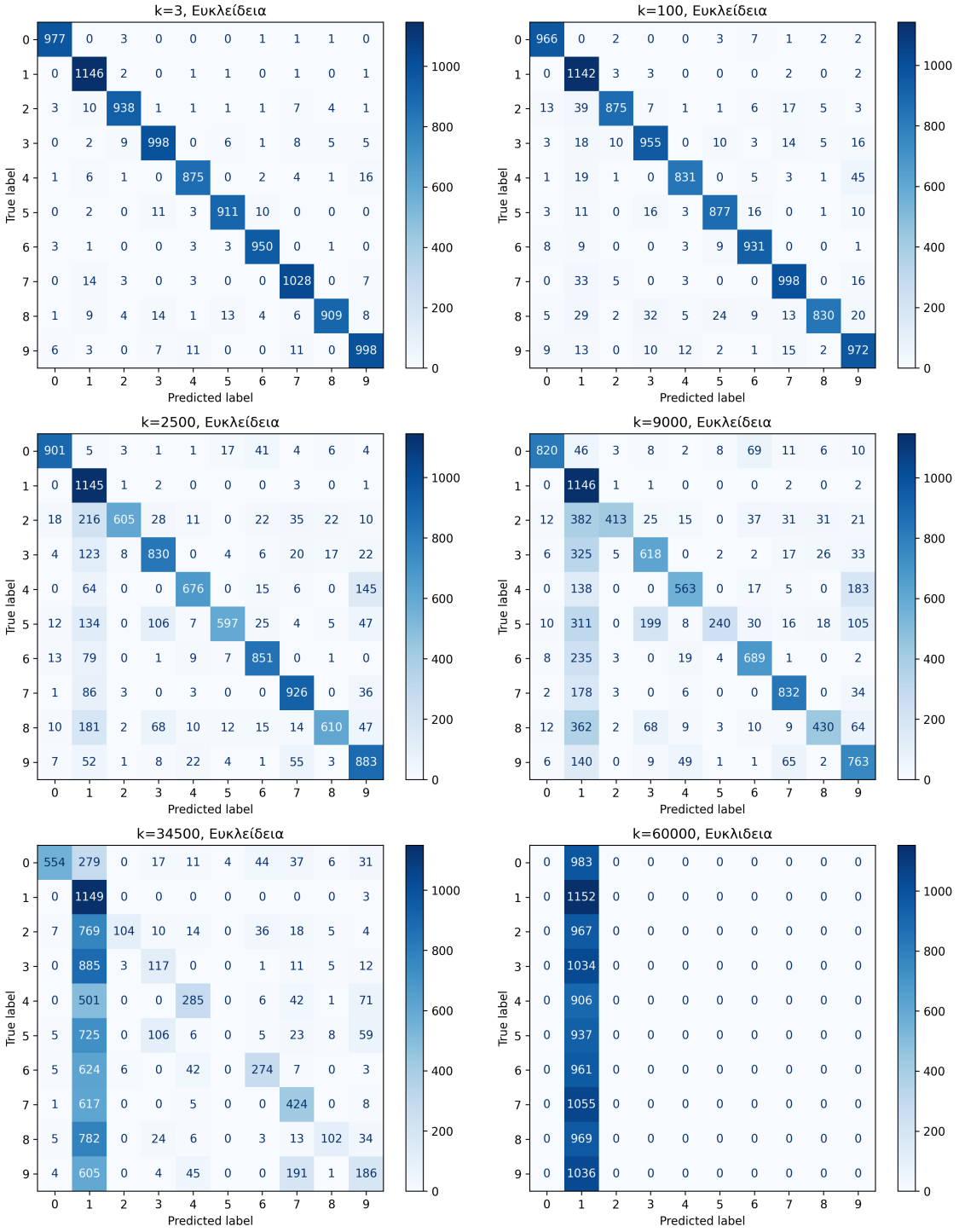

Figure 5: Confusion matrices for the k-NN algorithm using Euclidean distance for $k = 3, 100, 2500, 9000, 34500, 60000$. The gradual deterioration of predictions as k increases is observed.

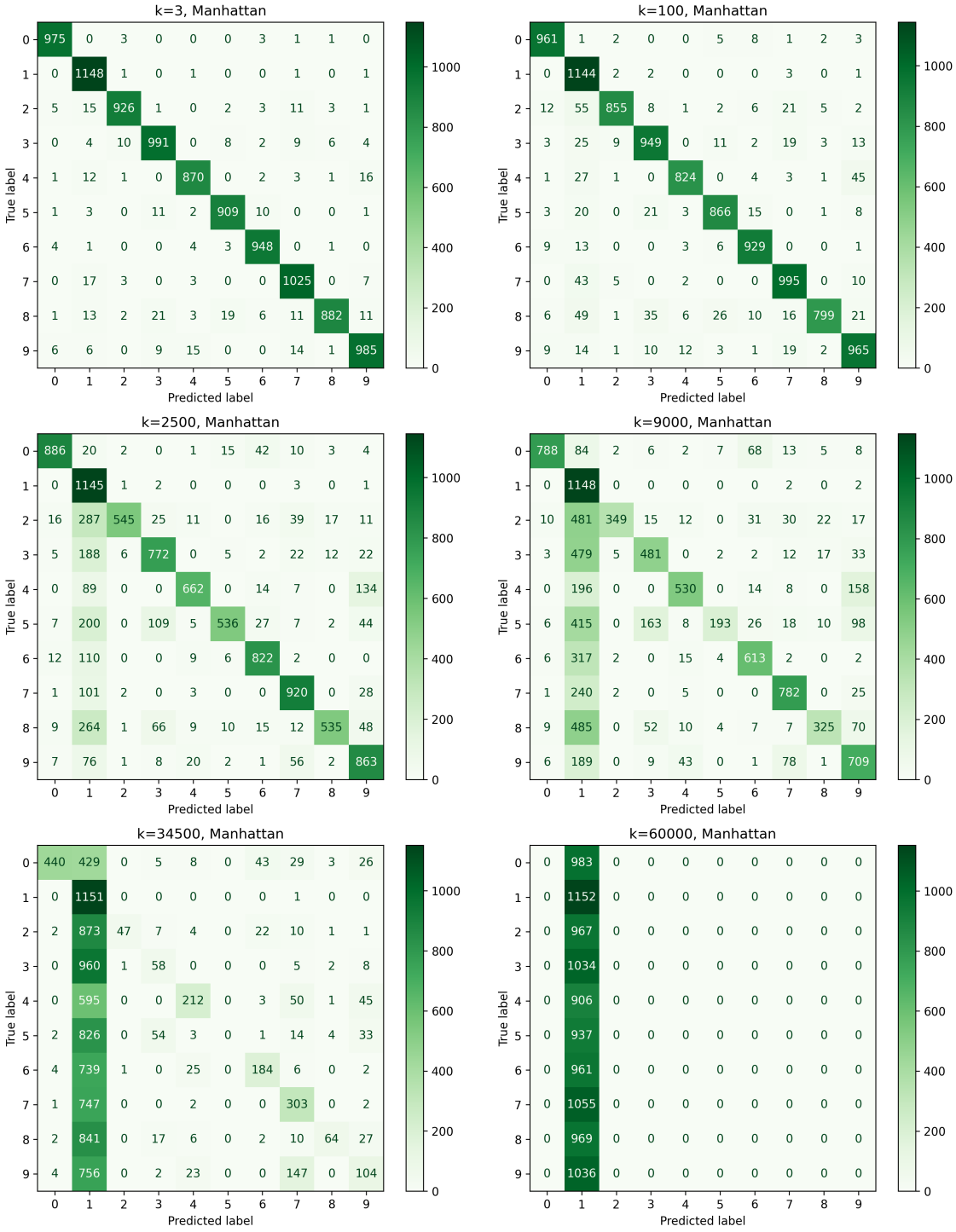

Figure 6: Confusion matrices for the k-NN algorithm using Manhattan distance for $k = 3, 100, 2500, 9000, 34500, 90000$. The gradual deterioration of predictions as k increases is observed.

- **Nearest Centroid classifier - Nearest centers**

  1. Euclidean (the center is the mean): 81.30% [cite: 1104]
  2. Manhattan (the center theoretically approaches the median): 65.76% [cite: 1105]

  Observing the results of Nearest Centroid, the following conclusions emerge[cite: 1106]:

  - **Superiority of Euclidean distance**: The Euclidean distance achieves a significantly higher accuracy (81.30%) compared to Manhattan (65.76%)[cite: 1107]. This happens because calculating the center as the arithmetic mean of the pixels (centroid) minimizes the sum of the squares of the distances (Euclidean geometry)[cite: 1108]. Conversely, the Manhattan distance operates optimally when the center is defined by the median of the values[cite: 1109].
  - **Model Limitations**: The lower performance of Nearest Centroid compared to k-NN is due to the fact that the former "condenses" all information of a class into a single mean image[cite: 1110]. In MNIST, where the same digit can be written in many different ways (e.g., the 7 with or without a crossbar), the mean creates a "blurry" representation that struggles to distinguish details, leading to frequent confusions between similar digits such as 4 and 9[cite: 1111].

  In Figure 7, the confusion matrices for the Nearest Centroid algorithm using Euclidean distance and Manhattan distance, respectively, are presented[cite: 1112]. The structure of these matrices is exactly the same as the structure of the matrices designed for the k-NN algorithm[cite: 1113].

Analyzing the form of the matrices, we clearly observe the superiority of the Euclidean distance[cite: 1114]. Specifically:

- **Euclidean distance**: Although the diagonal is clearly distinguished, the inherent weaknesses of the algorithm are revealed[cite: 1116]. Due to the center resulting from the mean of the images of a class, "blurry" patterns are created[cite: 1117]. This leads to characteristic confusions between digits that share common geometric features, as observed with '4' being frequently classified as '9' (115 times) or '5' as '3' (120 times)[cite: 1118].
- **Manhattan distance**: Here, a strong imbalance in predictions is observed, with the algorithm erroneously classifying a huge number of samples from various classes (such as '8', '2', and '3') into class '1'[cite: 1119]. This phenomenon is due to the mismatch between the way the center is calculated and the metric[cite: 1120]. The center (as an arithmetic mean) is optimized for the Euclidean distance[cite: 1121]. When the Manhattan distance is used, which sums the absolute differences of the pixels, the digit '1' is favored due to its "sparsity" (it has the fewest active pixels), thus incorrectly attracting many other digits[cite: 1122].

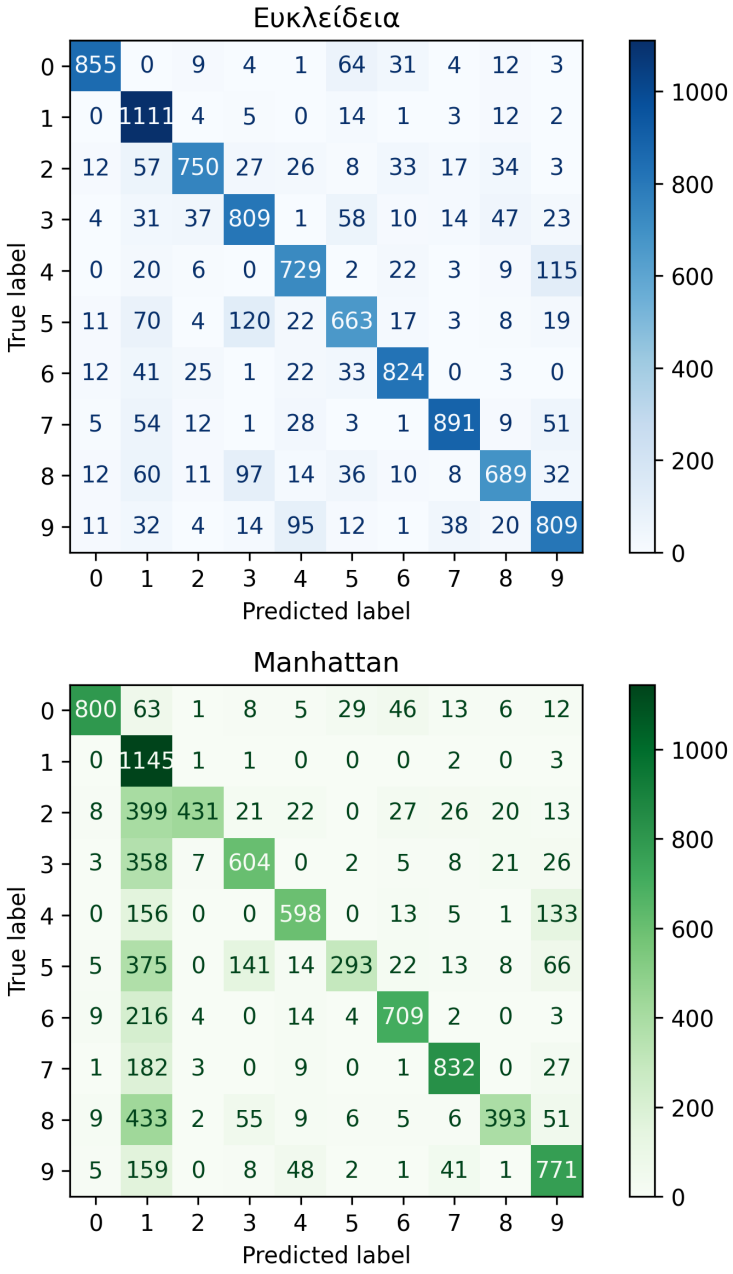

Figure 7: Confusion matrices for the Nearest Centroid classifier, using Euclidean distance and Manhattan distance.

Conclusions

Comparing the overall performance of the two algorithms on the MNIST dataset, the superiority of the k-NN classifier against the Nearest Centroid becomes clear. The k-NN (with $k = 3$) reached an accuracy of 97.30%, while the Nearest Centroid was limited to significantly lower levels (81.30%).

This difference is due to their way of operation. The Nearest Centroid calculates the mean of all samples of a class, creating a unique "pattern" for each digit. This approach is overly simple for handwritten digits, which present massive diversity in their way of writing (e.g., different slants, line thickness, decoration). Conversely, k-NN does not generalize the data into a center, but maintains the local information of the space, allowing it to successfully recognize multiple and different writing styles for the same digit, as long as the underfitting phenomenon is controlled by choosing an appropriately small k.